

22 Linux

[With contributions from Noah Boorstin, Ben Bracker, and Kaveh Pezeshki]

Booting a Linux operating system, in addition to being practical, is a good system test because it shows that all major subsystems of the processor work together, including the basic core, CSRs, exceptions, memory management, and peripherals. Linux comes in many distributions, and our application needs an embedded version that can start up from a small software image in memory, with only our basic peripherals and no disk, monitor, ethernet, or non-physical “virtual peripherals” common in software simulations. Wally has been tested using the free and popular Buildroot Linux generator.

While developing Wally, we have run Buildroot Linux on three platforms:

- A generic software RISC-V emulator QEMU
- Verilog simulation of the Wally SoC
- An FPGA implementation of the Wally SoC

This chapter demonstrates running Buildroot Linux in simulation on the QEMU and Wally platforms, and Chapter 23 shows how to run it on an FPGA implementation of Wally.

22.1 Principles

An OS has a variety of functions:

- Boot the computer when it turns on
- Protect hardware and other processes from faulty or malicious processes
- Provide resources to processes:
 - Process and thread management and scheduling
 - Memory management
 - File system management
 - Peripheral device drivers
 - Security
 - User interface

Contemporary operating systems for small computers can be classified as either *real-time operating systems* (RTOS) or *general-purpose operating systems*. At the time of this writing, RISC-V systems are mostly used in embedded systems and use no OS (“bare metal”), an RTOS, or embedded Linux.

Many RTOSes are on the market, such as FreeRTOS or VxWorks. They are generally invisible to the end user and little-known outside the developer community. They typically run on embedded systems, are designed to respond to inputs in a short and bounded amount of time, and are often optimized to run efficiently on inexpensive microcontrollers with small amounts of memory. RTOSes typically do not support virtual memory because a page fault causes a long and unpredictable delay, and because the system usually lacks secondary storage and enough memory to require complex management.

General-purpose operating systems, such as Linux, Windows, MacOS, or Android, are more widely known and visible to the user. Some run on PCs and mobile devices, but they are also used on embedded systems. Linux has become popular for complex embedded systems such as televisions or printers because it provides many powerful functions such as networking and scheduling, runs on most processors with MMUs, has a wide set of device drivers, is free and open-source, and has a large community of experts. Linux also has a real-time flavor called Preempt RT that guarantees a response within a bounded time and can serve some system that once required an RTOS. Linux has evolved to encompass more than 23 million lines of source code with contributions from more than 20,000 developers. As of 2023, about 68,000 lines are specific to supporting RISC-V.

Embedded Linux must be tailored to the particular system, including the processor architecture, peripherals, memory map, and software packages. *Open-source build systems*, also simply called *build systems*, automate this process. They produce a bootloader, kernel, and root file system. They often involve a cross-compiler, so they could run on an x86 PC or server but compile the OS to run on RISC-V or ARM. Contemporary build systems include Buildroot, Yocto, EmbToolkit, OpenEmbedded, OpnWrt, PTXdist, and Tizen. This section focuses on issues common to any Linux system, while Section 22.2 examines a Buildroot Linux system running on Wally RISC-V.

22.1.1 Embedded Linux Hardware Requirements

Embedded Linux runs on most 32- and 64-bit processor architectures with memory management units. It is easiest to configure with at least 16 MiB of RAM and 8 MiB of Flash memory storage, although it can be squeezed into smaller systems, and versions with many packages or lots of data need more storage. Embedded Linux usually needs a terminal interface, commonly RS-232 serial. It also needs a way to load software, such as an SD card, or USB, serial, or JTAG port.

22.1.2 Bootloader

When a processor first powers on or resets, it begins in the most privileged mode and sets the program counter to some address called the *reset vector*. The system must have a small ROM or Flash memory at this address containing *zero stage bootloader* (ZSBL) code to start the boot process. The DRAM controller is not yet initialized, so the boot loader initially has access only to

a small on-chip SRAM and the boot ROM. Booting is typically divided into several phases, each of which brings up more capabilities for the system and transfers code for the next phase from nonvolatile storage into RAM until the boot loader can load the Linux kernel into DRAM and transfer control to it [Mastering Embedded Linux Programming]. The details are platform-specific. Common bootloaders include GRUB for PCs and U-Boot for embedded systems.

<SIDEBAR>

The term bootloader gets its name from the expression “pulling yourself up by your bootstraps,” or humorously lifting yourself up into the air by pulling up on your own boots. The process of starting to run an OS without having the capabilities of an OS to run code is a similar challenge. The expression is now shortened to “booting” a computer.

</END>

22.1.3 Device Tree

During the boot process, the bootloader or OS must identify what hardware devices are in the system. For example, it needs to determine how much RAM is available and what nonvolatile storage device contains the kernel and filesystem. Later, it needs to determine what peripherals are available, such as a serial port to print to a console, or a timer to generate interrupts. It needs to identify where these peripherals are in the memory map, and what version of the hardware exists so the OS can use the corresponding device driver.

Historically, this information was compiled into the kernel, such that changing the hardware could require recompiling the OS. In the 1980s, Sun Microsystems defined the Open Firmware specification including a *device tree*, which is a data structure that describes the hardware configuration, including:

- Number and type of CPU cores
- Amount of DRAM
- Types and versions of peripherals, such as serial ports, timers, nonvolatile storage devices, and interrupt controllers
- Memory map with addresses for the memory and peripherals
- Mapping of peripherals to interrupt lines

Device trees are now used by almost all Linux platforms. They are described by human-readable text files with a .dts suffix. A device tree compiler converts them to a more compact binary format called a *device tree blob* or *flattened device tree* (FDT) with a .dtb suffix. Typically, the bootloader loads the FDT and passes it to the kernel.

22.1.4 Kernel

The *kernel* is the portion of the OS software that operates in privileged mode, providing an interface between the hardware and user processes. It is responsible for managing memory, processes, device drivers, and system calls.

One of the principles of Linux is that a common kernel image can run on a wide variety of hardware with different numbers and types of cores and peripherals, without requiring recompilation. The kernel calls an underlying firmware layer that provides a standardized interface to platform-dependent tasks such as starting and stopping cores. It uses the device tree to define the set of peripherals. It includes device drivers as kernel modules that can be loaded from the `/dev` directory in the file system and executed with the same privileges as the OS.

<SIDEBAR>

Linus Torvalds maintains the *mainline* Linux kernel and releases new kernels every 9-10 weeks. Contributors develop new features that are integrated into release candidate builds and tested before becoming part of the mainline kernel. A *longterm maintenance* kernel is released about once a year, and just receives critical bug fixes rather than constant new features. Linux distributors such as Ubuntu and Red Hat integrate the kernel with the rest of the OS (such as the U-mode shell and graphical user interface) and release periodic distributions and unscheduled bug fixes.

</END>

The kernel is often kept in nonvolatile storage in compressed format. Once loaded, it decompresses itself, enables virtual memory, and calls `start_kernel()`, which in turn initializes device drivers, enables interrupts, mounts a root file system, and then starts the idle process, scheduler, and init process.

The kernel may exist in various file formats in the `/boot` directory:

- `vmlinux`: The kernel is in a statically linked ELF; suitable for disassembling or debugging
- `vmlinuz`: `vmlinux` compressed with `zlib`
- `Image`: `vmlinux` in binary format, with ELF headers and symbol table removed, suitable for booting directly
- `zImage`: `Image` compressed with `zlib`

22.1.5 Root Filesystem

The Linux kernel loads a *root filesystem* into RAM containing a minimal set of non-kernel programs necessary to boot. The filesystem is called `rootfs` or `initramfs`. It is kept in `.cpio` (Copy In and Out) format archive. The archive may be compressed with `gzip` to save storage space and time to read in from the storage device, so the kernel may uncompress the file system when loading it into memory.

Core Linux utilities such as `ls`, `cp`, and `init` collectively take a fair amount of disk space. To reduce the size of the root filesystem, small embedded Linux systems often use a program called BusyBox that performs most of the basic functionality of these utilities while being much smaller (e.g., ~6 MB) by leaving out unnecessary features and sharing code across utilities. Each of the regular utilities in `/bin` is actually a symbolic link to `/bin/busybox`, which interprets the command line and performs the functions.

A fully-featured Linux system will use the drivers and utilities in `rootfs` to read a larger file system from disk later in the boot process, replacing `rootfs`.

22.1.6 Init

The first U-mode process is called `init` (short for initialization). It has a process ID of 1 and is the parent of all other Linux processes. It is called a *daemon* process, which runs in the background independently of the control of interactive users. The exact steps taken by `init` vary from system to system and continue to evolve. The Busybox `init` program reads a configuration file called `/etc/inittab` that contains a list of programs to run at startup time. These programs include daemons that provide services such as `nfsd` (Network File System Server), `sshd` (Secure Shell Server), `httpd` (Hypertext Transfer Protocol Server), or `lpd` (Printer Server). `systemd` is a more modern `init` daemon that starts processes in parallel to reduce boot time.

<SIDEBAR>

Daemons get their name from *Maxwell's daemon*, an imaginary being in thermodynamics that constantly sorts molecules in the background. *daemon* is pronounced “dee-mon” not “day-mon”.

</END>

22.1.7 Shell

`init` typically then launches a login *shell* with job control so that the user can log in, issue commands, and use <Ctrl-C> to terminate a running program. Linux offers various shells, with `bash` currently being the default on most distributions. `ash` is a lightweight version of `bash` provided by Busybox.

22.2 Buildroot Linux on Wally

Although it is possible to roll your own Linux system manually, it is faster and more maintainable to use an automated system to build these from source code in a configurable way. The Buildroot and Yocto build systems are among the most popular [Mastering Embedded Linux Programming]. We use Buildroot, which supports RISC-V and produces the OpenSBI bootloader/firmware, Linux kernel, and root filesystem for embedded Linux. This section applies

the general principles from Section 22.1 to the specific case of booting Buildroot Linux on RISC-V for Wally, including the bootloaders, device tree, kernel, and root file system. Section 22.2.1 gives directions for configuring, building, and disassembling Buildroot in the `$RISCV` directory using QEMU. The remainder of this section examines the detailed process of booting Linux on Wally. Section 22.3 gives detailed instructions for running Buildroot and booting Linux in QEMU and Verilog simulation. Section 23.2.2.4XREF illustrates booting Linux on an FPGA.

This section first defines where all the source, binary, and disassembled files can be found and where they are loaded into physical and virtual memory. Then it summarizes key steps in the boot process. Finally, it explores the ZSBL, OpenSBI, kernel, file system, and device tree in more depth.

The specific examples in this section are based on Buildroot 2023.05.3-dirty. They may have changed if you rebuild with a newer version.

22.2.1 Building Buildroot Linux

To download, cross-compile, disassemble, generate memory files for, and install Buildroot Linux for RISC-V into `$RISCV/buildroot`, use the `$WALLY/linux/Makefile`. Compilation takes approximately 10 minutes. If necessary, you will be prompted for your password for sudo permissions to install buildroot in `$RISCV`. The `$WALLY/bin/wally-tool-chain-install.sh` script will automatically make buildroot while installing everything else. Section 22.2.2.4 will describe how to generate a trace of the instructions executed during the Linux boot.

```
$ cd $WALLY/linux # or your path to the repository
$ make           # clone, build, disassemble linux and install in $RISCV
```

22.2.2 Source, Binary, and Disassembled Files

Table 22.1 defines the directories where all key files are placed.

Table 22.1 Linux file locations

Files	Path
Source Code	<code>\$RISCV/buildroot/output/build</code>
Binary Files & Device Tree	<code>\$RISCV/buildroot/output/images</code>
Disassembled Files	<code>\$RISCV/buildroot/output/images/disassembly</code>
Memory Files	<code>\$RISCV/linux-testvectors</code>

Table 22.2 summarizes the components of the boot process. Each is discussed briefly here and in more detail in subsequent subsections. The zero stage bootloader (ZSBL) is a few bytes in ROM at address 0x1000. In a real system, it would load more code from nonvolatile storage, but in this system, it just jumps to the OpenSBI firmware already in memory. This firmware contains the rest of the bootloader, which initializes the hardware (informed by the device tree) and M-mode traps, then in turn jumps to the Linux kernel. The kernel enters S-mode, turns on virtual memory, and initializes services. It then loads init from the root file system on the RAM disk and starts it as the first U-mode process.

Table 22.2 Linux boot components

Component	Source Code Subdirectory	Binary File	Disassembly
Zero Stage Bootloader	\$WALLY/linux/bootmem.txt	\$RISCV/linux-testvectors/bootmem.bin	n/a
OpenSBI Firmware	opensbi-1.3	fw_jump.bin	fw_jump.objdump
Linux Kernel	linux-6.6	Image	vmlinux.objdump
Root Filesystem	n/a	rootfs.cpio	uncompressed into rootfs
Device Tree	\$WALLY/linux/devicetree/wally-virt.dts	wally-virt.dtb	n/a
Init	busybox-1.36.1	\$ROOTFS/sbin/init -> /bin/busybox in rootfs	busybox.objdump

22.2.2.1 Binary Files

The ZSBL is not stored in a file but is placed in memory by QEMU. It is shown in Section 22.2.5.

Building Buildroot produces the firmware and Linux kernel in both ELF format (fw_jump.elf / vmlinux) and raw binary format (fw_jump.bin / Image). The ELF versions include headers and debug information suitable for disassembly, while the binary versions are loaded directly into memory by QEMU.

The root file system is stored as a compressed binary rootfs.cpio file also loaded directly into memory by QEMU. Compression speeds up reading the file from a slow flash memory device, at the cost of CPU operations to decompress the file.

Wally's device tree wally-virt.dts is compiled into a flattened device tree blob wally-virt.dtb and is also preloaded into memory.

22.2.2.2 Disassembled Files

After running make disassembly, \$RISCV/buildroot/output/images/disassembly contains:

- fw_jump.objdump OpenSBI firmware
- vmlinux.objdump Linux kernel

- `busybox.objdump` BusyBox
- `rootfs` Uncompressed root file system (see Section 22.2.8)

The flattened device tree blob is not disassembled because it is not an executable file and the raw `.dts` file is available.

22.2.2.3 Memory Files

When simulating Linux on Wally, `testbench.sv` preloads the boot ROM at 0x1000 and uncore RAM at 0x80000000 with binary files `bootmem.bin` and `ram.bin`, respectively. `bootmem.bin` is the trivial ZSBL described in Section 22.2.5, and `ram.bin` is a 256 MiB memory image containing the firmware, kernel, and root file system.

Running `make dumptvs` uses QEMU to generate these memory images into `$RISCV/linux-testvectors`. The top-level Linux make command runs this automatically.

22.2.2.4 Boot Trace

To understand the behavior of the Linux boot, it is helpful to have a trace showing the sequence of instructions executed from power-on reset until Linux is waiting at the command prompt. First, the tracer script must be modified to enable logging. Then run a Questa simulation of the Linux boot in lock step with ImperasDV and record the trace into `$WALLY/sim/logs/boottrace.log`.

```
# edit testbench/common/wallyTracer.sv at line 26 to `define STD_LOG 1
$ wsim buildroot buildroot --lockstep
```

`boottrace.log` takes more than a day to generate and is about 51 GiB in size. Instructions causing exceptions are not listed because they don't commit.

Code Example 22.1 shows the first instructions in the trace, starting with the bootloader at 0x1000, then jumping into OpenSBI at 0x80000000, and then to main at 0x80000578.

Code Example 22.1 Beginning of `boottrace.log`

```
0000000000001000, 00000297, AUIPC    r[ 5] = 0000000000001000
0000000000001004, 02828613, ADDI    r[12] = 0000000000001028 csr[b00] = 000000000000000f csr[b02] = 0000000000000002
0000000000001008, f1402573, CSRRES r[10] = 0000000000000000 csr[b00] = 0000000000000001a csr[b02] = 0000000000000003
000000000000100c, 0202b583, LD      r[11] = 000000008fe00000 csr[b00] = 0000000000000001b csr[b02] = 0000000000000004
0000000000001010, 0182b283, LD      r[ 5] = 0000000080000000 csr[b00] = 0000000000000001c csr[b02] = 0000000000000005
0000000000001014, 00028067, JALR    r[ 0] = 0000000000000000 csr[b00] = 00000000000000028 csr[b02] = 0000000000000006
0000000080000000, 00050433, ADD     r[ 8] = 0000000000000000 csr[b00] = 0000000000000002b csr[b02] = 0000000000000007
0000000080000004, 000584b3, ADD     r[ 9] = 000000008fe00000 csr[b00] = 0000000000000002c csr[b02] = 0000000000000008
0000000080000008, 00060933, ADD     r[18] = 0000000000001028 csr[b00] = 00000000000000037 csr[b02] = 0000000000000009
000000008000000c, 56c000ef, JAL     r[ 1] = 0000000080000010 csr[b00] = 00000000000000038 csr[b02] = 000000000000000a
0000000080000578, 8082557d, C.LI    r[10] = ffffffff csr[b00] = 0000000000000003b csr[b02] = 000000000000000b
```

22.2.3 Linux Memory Map

Wally uses the memory map from Figure D.2XREF, including a small ROM at 0x1000 for the ZSBL (zero stage bootloader) and a 128 MiB RAM at 0x80000000-87FFFFFF. In QEMU emulation or SystemVerilog simulation, the emulator or testbench places the code, root file system, and device tree into memory before the boot process begins, and then a trivial ZSBL jumps to the firmware, as described in Section 22.2.5. On an FPGA, the ZSBL reads these files from a mass storage device such as a SD flash memory card, as described in Section 23.XREF.

Observe that the kernel is at physical address 0x80200000 even though it is compiled assuming virtual address 0xFFFFFFFF80000000. Hence, the PC will be different running the kernel before and after virtual memory is turned on.

Table 22.3 Linux address space layout

Component	Physical Address	Binary Size
Zero Stage Bootloader	0x1000	72 B
OpenSBI Firmware	0x80000000	146 KiB
Linux Kernel	0x80200000	72 MiB
Root Filesystem	0x88200000	60 MiB
Device Tree	0x8fe00000	2 KiB

The kernel runs in S-mode and is placed in virtual memory at a high address with all leading 1s (0xFFFFFFFF80000000). When the kernel needs to access bare hardware, it calls the OpenSBI supervisor binary interface [<https://github.com/riscv-non-isa/riscv-sbi-doc/blob/master/riscv-sbi.adoc>], which runs in M-mode and gives a standardized supervisor execution environment (SEE) for the OS to access the platform-dependent raw hardware, such as changing the timer, writing a character to a console, or starting, stopping, or synchronizing harts. The SBI is also called *firmware*, and runs in M-mode starting at physical memory address 0x80000000. Linux programs run in U-mode and are sometimes loaded into virtual memory at low addresses.

22.2.4 Linux Boot Overview

Linux boot involves executing about half a billion instructions, although about 80% of them are for decompressing the root filesystem. Table 22.4 summarizes the major events during the boot process in QEMU. The instruction number, address, and instruction come from `boottrace_disasm.log`. The disassembly column lists the file containing the disassembled code for context. Search for the address in the disassembly.

<SIDEBAR>

Beware that the trace includes instructions that trapped and didn't commit, so the instruction number can exceed instrret.

</END>

<SIDEBAR>

These milestones can be found with
grep -n address boottrace_disasm.log

where address is the number (without 0x) in the address column. Generally, the first find is the interesting one.

</END>

Table 22.4 Linux boot milestones

Instruction #	Address	Instruction	Source	Disassembly	Description
1	0x1000	auipc t0, 0		bootmem.txt	Reset Vector: Start of bootloader
6	0x1014	jr t0		bootmem.txt	Jump to SBI Firmware
189,190	0x800003d8	csrw mtvec,a4	firmware/fw_base.S _start_warm	fw_jump.objdump	Enable Machine Trap Handler called _trap_handler
875,039	0x800088a6	<mstatus_init>	Sbi/sbihart.c mstatus_init()	vmlinux.objdump	Enable features: FPU, counters, CMO, Svpbmt,
875,488	0x80008a44	csrw mideleg, 222	lib/sbi/sbi_hart.c delegate_traps()	fw_jump.objdump	Delegate all supervisor interrupts, page faults, U-mode ecall, and ebreak to supervisor
1,286,518	0x800008d0	jal <sbi_printf>	lib/sbi/sbi_init.c sbi_boot_print_banner()	fw_jump.objdump	Print SBI Boot Message
5,041,679	0x8000aa8e	mret	lib/spi/sbi_hart.c sbi_hart_switch_mode()	fw_jump.objdump	Enter S-Mode and start kernel at 0x80200000
5,187,363	0x80201044	csrw satp,a0	arch/riscv/kernel/head.S relocate_enable_mmu	vmlinux.objdump virtual address ffffff80001044	Turn on Virtual Memory, causing page fault that enters kernel at fffffff80001048
5,187,374	0xffffffff8000106c	csrrw stvec, a0	arch/riscv/kernel/head.S relocate_enable_mmu	vmlinux.objdump	Enable Supervisor Trap Handler to handle exception
12,344,404	0xffffffff80400b0a	csrrsi sstatus,2	init/main.c start_kernel()	vmlinux.objdump	Turn on S-Mode Interrupts
12,416,620	0xffffffff80247f5c	<riscv_timer_interrupt>	drivers/clocksource/timer-riscv.c riscv_timer_interrupt()	vmlinux.objdump	First timer interrupt
20,500,509	0xffffffff804023c8	<populate_rootfs>	init/initramfs.c populate_rootfs()	vmlinux.objdump	Populate rootfs
424,172,631	0xffffffff80270f94	<run_init_process>	init/main.c run_init_process()	vmlinux.objdump	Start Init
424,516,404	0xffffffff800031c8	sret		vmlinux.objdump	Enter U-Mode

					program at 0x00007fff9ca04c3c
492,486,904	0x65254	<print_login_prompt>	libbb/login.c print_login_prompt()	busybox.objdump	Print Login Prompt
492,515,478	0xffffffff800033da	wfi	kernel/sched/idle.c do_idle()	vmlinux.objdump	Idle Loop

Note that `boottrace.log` and `boottrace_disasm.log` are too large to open in most text editors. The best way to find a line of code in context is with `grep`, including the `-C` flag to list lines of code before and after, and the `-n` flag to list line numbers. If you need more context, `sed` will print a set of lines. For example, to find the code that enables the machine trap vector:

```
$ grep mtvec -C 2 -n $WALLY/linux/testvector-generation/boottrace_disasm.log
189188-00000000800003d0, 00000717,"auipc    a4,0x0"      rf[14] = 00000000800003d0
189189-00000000800003d4, 05870713,"add    a4,a4,88"      rf[14] = 0000000080000428
189190-00000000800003d8, 30571073,"csrr    mtvec,a4"      rf[ 0] = 0000000000000000
189191-00000000800003dc, 34002573,"csrr    a0,mscratch"    rf[10] = 0000000080027000
189192-00000000800003e0, 1fc000ef,jal    922b8 <main+0x922b8  rf[ 1] = 00000000800003e4

$ grep 800003d8 $RISCV/buildroot/output/images/disassembly/fw_jump.objdump
800003d8:          30571073          csrr      mtvec,a4

$ sed -n '189185,189190p' $WALLY/linux/testvector-generation/boottrace_disasm.log
00000000800003c4, 40f20233,"sub    tp,tp,a5"          rf[ 4] = 0000000080027000
00000000800003c8, 34021073,"csrr    mscratch,tp"        rf[ 0] = 0000000000000000
00000000800003cc, 00020133,"add    sp,tp,zero"      rf[ 2] = 0000000080027000
00000000800003d0, 00000717,"auipc    a4,0x0"      rf[14] = 00000000800003d0
00000000800003d4, 05870713,"add    a4,a4,88"      rf[14] = 0000000080000428
00000000800003d8, 30571073,"csrr    mtvec,a4"      rf[ 0] = 0000000000000000
```

22.2.5 Zero Stage Bootloader

A RISC-V processor boots in M-mode. Although the RISC-V reset vector is nominally platform-dependent, Spike, QEMU, the SiFive FU540 (and Wally) start the PC at 0x1000 and execute a short segment of code that jumps to another address (typically 0x80000000).

QEMU (`riscv_setup_rom_reset_vec` in `hw/riscv/boot.c`) places the ZSBL from Code Example 22.2 in memory at 0x1000, and the Wally testbench loads the same code from `bootmem.bin` for compatibility. This bootloader sets `a0` to the `mhartid`, `a1` to the address of the device tree, `a2` to a data structure that includes the Linux kernel address, and then jumps to the OpenSBI firmware at 0x80000000.

Code Example 22.2 Zero stage bootloader (ZSBL)

```
00001000: 00000297          auipc    t0, 0          # t0 = 0x00001000
00001004: 02828613          addi    a2, t0,0x28      # a2 = 0x00001028
00001008: f1402573          csrr    a0, mhartid     # a0 = mhartid
0000100c: 0202b583          ld      a1, 32(t0)      # a1 = 87000000 - device tree address
00001010: 0182b283          ld      t0, 24(t0)      # t0 = 80000000 - start of firmware
00001014: 00028067          jr      t0             # jump to firmware
00001018: 0000000080000000          # firmware start address
00001020: 000000008fe00000          # flattened device tree load address
00001028: 000000004942534f          # a2 points to this 8 dword data structure
00001030: 0000000000000002
```

22.2.6 OpenSBI Firmware

The OpenSBI firmware also boots in M-mode and is responsible for initializing the hardware.

The firmware supervisor binary interface (SBI) also provides a level of abstraction between the kernel (*e.g.*, calls to `console_putc()` to write a character to the console) and the hardware (*e.g.*, the underlying UART registers) so that the kernel can run on many different hardware *platforms* without modification. Thus, the firmware can be configured with the appropriate platform, such as the SiFive FU540 system-on-chip or an Ariane FPGA. Wally is configured to use a generic platform that examines the flattened device tree to determine which peripherals are available. See `$WALLY/linux/buildroot-config-src/wally/main.config`
`BR2_TARGET_OPENSBI_PLAT="generic"`.

By default, OpenSBI assigns all the harts and memory to a *root domain*. The firmware can optionally partition the system into multiple *domains* specified in the device tree such that the harts and memory associated with each domain are isolated from each other via physical memory protection. These domains are a hardware foundation for features such as trusted execution environments (TEEs), secure operating systems, and root-of-trust.

[https://static.sched.com/hosted_files/riscvsecurityforum2021/84/OpenSBI_domain_support_v4.pdf]

Each hart has scratch memory space that contains firmware data structures. The `mscratch` register points to this memory. The `sbi_scratch` structure includes pointers to the firmware, kernel, and the `sbi_platform` structure. The `sbi_platform` structure contains the name and version number of the platform, the number of harts, and a pointer to the `sbi_platform_operations` structure. The `sbi_platform_operations` structure includes pointers to platform-specific functions for initializing and using the timer, console (UART), inter-processor communication software interrupts, and IRQ chip external interrupts.

The initial assembly language bootloader code is at `_start` in `firmware/fw_base.S`. All RISC-V harts power up in random order, but the initial firmware boot occurs on only one hart. Thus, one of the harts is selected through a lottery (accessing shared memory) to be the boot hart, while the others sleep. Assembly language boot steps include:

- Initialize registers and BSS memory to zero
- Set up temporary stack and trap handler to hang if boot fails
- Read platform information from the flattened device tree
- Point `mscratch` to initialized scratch space with per-hart firmware data structures
- Wake up the other harts by writing a flag to a `_boot_status` memory location
- Disable and clear interrupts

- Set up sp stack pointer and MTVEC trap handler `_trap_handler`

The bootloader then calls `sbi_init` in `lib/sbi/sbi_init.c`. It reads the flattened device tree (FDT) to determine which types of peripherals are available and has simple drivers to initialize essential peripherals. It then calls a series of initialization functions:

- `sbi_scratch_init` Scratch space for firmware data structures
- `sbi_heap_init` Heap memory
- `sbi_domain_init` Root domain containing assigned memory and harts
- `sbi_hsm_init` Hart state management: start/stop/suspend
- `sbi_hart_init` Hart features, init mstatus, fp, trap delegation
- `sbi_console_init` Configure UART baud rate and control registers
- `sbi_boot_print_banner` Display boot messages
- `sbi_irqchip_init` Disable external interrupts
- `sbi_ipi_init` Configure software (inter-process) interrupts
- `sbi_tlb_init` Set up cross-hart TLB flush
- `sbi_timer_init` Configure timer
- `sbi_hart_pmp_configure` Set up physical memory protection for domains
- `sbi_ecall_init` Register SBI extensions
- `sbi_boot_print_general` Display more boot messages
- `sbi_boot_print_domains` Domain-specific boot messages
- `sbi_boot_print_hart` Hart-specific boot messages
- `wake_coldboot_harts` Send IPI message to other harts waiting for boot
- `sbi_hart_switch_mode` Enter the Linux kernel in S-mode

22.2.6.1 UART Initialization

To illustrate hardware initialization, consider the `sbi_console_init` UART initialization.

Wally uses the generic platform, which examines the flattened device tree to find the system has an ns16550a-compatible UART (see Section 20.4XREF) at a clock frequency of 3.6864 MHz. The device tree does not specify the baud rate, so it defaults to 115200 in `lib/utis/fdt/fdt_helper.c`. `lib/utis/serial/fdt_serial_uart8250.c` assigns this UART to the UART8250 driver. `lib/utis/serial/uart8250.c` initializes the UART with a sequence of accesses given in Table 22.5.

Table 22.5 UART initialization accesses

Address	Value	Register	Purpose
10000001	00	Interrupt Enable	Disable Interrupts
10000003	80	Line Control	Enable Divisor Latch Access Bit (DLAB)

10000000	02	Divisor Latch Low	Set baud rate to 115200
10000001	00	Divisor Latch High	Set baud rate to 115200
10000003	03	Line Control	8 data bits, 1 stop bit, no parity, disable DLAB
10000002	01	FIFO Control	Enable FIFO
10000004	00	Modem Control	Disable modem control signals such as CTS
10000005	Read	Line Status	Clear all line status errors
10000000	Read	Receiver Buffer	Clear any data sitting in receive buffer
10000007	00	Scratchpad	Clear UART scratchpad

The UART is now ready to communicate. The firmware `console_putc` and `console_getc` routines invoke `uart8250_putc` and `uart8250_getc`. `putc` waits until the Line Status Register (LSR) indicates the Transmit Holding Register (THR) is empty and then writes the character to the THR. `getc` waits until the LSR indicates that data is ready and then reads the Receiver Buffer Register.

22.2.6.2 PLIC Initialization

As another example of hardware initialization, consider `sbi_platform_irqchip_init` to configure external interrupts on the PLIC. The device tree indicates the PLIC has 53 sources and is compatible with `riscv,plic0`, so the firmware uses the driver in `lib/utils/irqchip/fdt_irqchip_plic.c` and `plic.c`.

The driver first calls `plic_cold_irqchip_init`, which sets a priority of 0 for all 53 interrupt sources, disabling them. The driver then calls `plic_warm_irqchip_init`, which clears the interrupt enable bits for all 53 sources in contexts 0 (machine) and 1 (supervisor) and also sets the priority threshold to 7 for both contexts to disregard all interrupts.

22.2.6.3 PMP Configuration

The `sbi_hart_pmp_configure` function sets up the PMP physical memory protection registers to isolate domains. Wally uses a single domain, so there are only two PMP regions, one to protect the firmware from lower-privileged access, and the other granting universal access to all other memory.

The PMP registers are configured with `pmpadr0 = 0x2003fff`, `pmpadr1 = 0xfffffffffffff`, and `pmpcfg0 = 0x1f18`. This sets regions 0 and 1 to naturally aligned power-of-two (NAPOT) sizes. Region 0 spans physical addresses `0x80000000-0x8001ffff`, encompassing the SBI. The access bits are all 0, so this 128-KiB SBI memory space is secured against read, write, and execute access by S- and U-mode programs. Region 1 spans all of physical memory and has read, write, and execute access, so all programs can access any other part of physical memory excluding region 0. Notably, the S-mode kernel has direct access to I/O peripherals. The kernel prohibits

mapping U-mode virtual addresses to peripheral physical addresses, so U-mode programs must access peripherals through OS calls rather than by direct memory-mapped I/O.

22.2.7 Kernel

The kernel runs in S-mode. It has some subtleties transitioning from firmware to kernel and from physical addressing to virtual addressing. The kernel is loaded at physical address 0x80200000, which will be mapped to virtual address 0xFFFFFFFF80000000. The firmware `sbi_hart_switch_mode` function enters the kernel by placing 0x80200000 in the `epc`, 01 (for S-mode) in `mstatus.MPP`, and executing an `mret` to jump to the start of the kernel and enter S-mode.

The assembly language kernel startup code is in `arch/riscv/kernel/head.S`. It begins at `_start` and immediately jumps to `_start_kernel`. Early steps include:

- Turn off interrupts
- Initialize the global pointer a location above the Linux kernel
- Disable the FPU to detect illegal use of floating-point operations in the kernel
- Hold a lottery to pick one hart to run the boot

`head.S` then calls `setup_vm` to create initial page tables mapping the virtual addresses in the kernel starting at 0xFFFFFFFF80000000 to the physical addresses where the kernel is loaded at 0x80200000, using megapages for efficiency. It then calls `relocate_enable_mmu`, listed in Code Example 22.3, which writes `satp` to enable virtual memory. The instruction fetch immediately after the `satp` write causes a page fault because the `PC = 0x80201054` is not a valid virtual address. The trap vector `stvec` is preset to the next instruction in the program, but at the corresponding virtual address 0xffffffff80001054, so the program continues properly. This switch from machine physical addressing to supervisor virtual addressing is called a *trampoline* because the trap handler immediately bounces the program into the kernel code. Now the kernel can load the rest of the kernel page tables, call `setup_trap_vector` to point `stvec` to the `handle_exception` supervisor trap handler, and eventually call `start_kernel` in `init/main.c`.

Code Example 22.3 Linux kernel enabling virtual memory (`head.S: relocate_enable_mmu`)

```
relocate_enable_mmu:
...
0000000080201020: 10561073 csrw stvec,a2      # STVEC = ffffffff80001048 right after SATP write
...
0000000080201040: 12000073 sfence.vma         # Ensure new page tables are in use
0000000080201044: 18051073 csrw satp,a0        # SATP = 90000000000848b2 (RV48 root)
                                # use trampoline page table at 0x848b2000
                                # cause page fault to STVEC (next instruction)
0000000080201048: 00000517 auipc a0,0x0        # a0 = ffffffff80001048
000000008020104c: 03450513 add a0,a0,52        # a0 = ffffffff8000107c (a spin loop)
0000000080201050: 10551073 csrw stvec,a0      # point STVEC to ffffffff8000107c to spin for debug
ffffffff80001054: 046a8197 auipc gp,0x46a8    # update global pointer to virtual addr
```

```

ffffff80001058: ee418193 add gp, gp, -284 # gp = fffffff846a8f38
ffffff8000105c: 18061073 csrw satp, a2 # SATP = 900000000080805 (RV48 kernel page table)
ffffff80001060: 12000073 sfence.vma # switch to the new page table
ffffff80001064: 8082 ret # return to _start_kernel
ffffff800010ee: f7bff0ef jal setup_trap_vector
setup_trap_vector:
ffffff80001068: 00002517 auipc a0, 0x2 # a0 = fffffff80003068
ffffff8000106c: 02450513 add a0, a0, 36 # a0 = fffffff8000308c <handle_exception>
ffffff80001070: 10551073 csrw stvec, a0 # STVEC = fffffff8000308c (handle_exception)
ffffff80001074: 14001073 csrw sscratch, zero # SSCRATCH = 0 indicates kernel code
ffffff80001078: 8082 ret # return to _start_kernel

```

`start_kernel` invokes many more initializations including multiprocessing, memory management, scheduling, interrupts, timers, and the console. The details are beyond the scope of this book.

22.2.8 Root File System (rootfs)

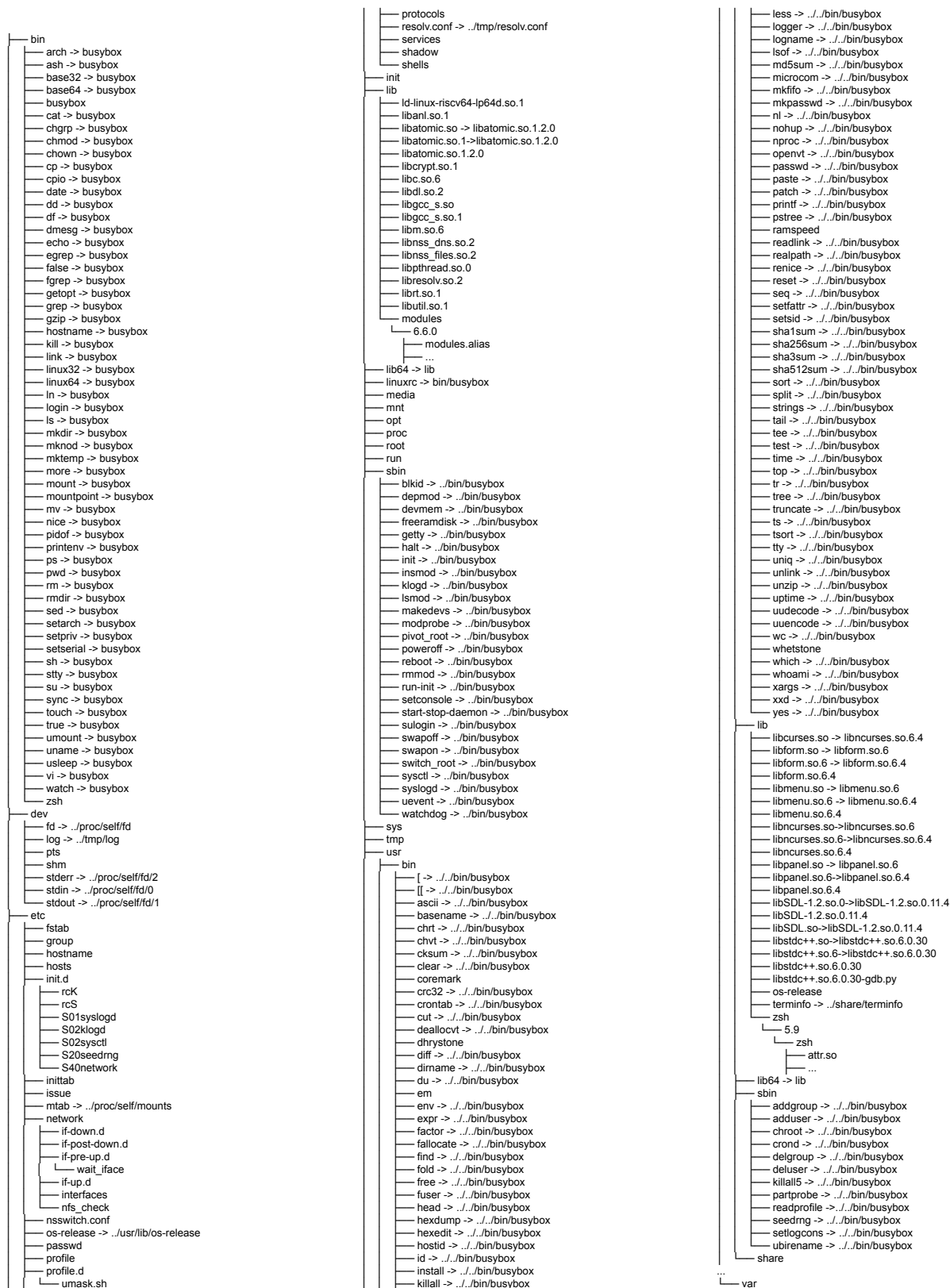
The compressed root file system is placed in memory as `rootfs.cpio`. The file system is listed in Figure 22.1. All the executable commands are symbolic links to the BusyBox root file system which parses the command line and executes the appropriate command.

<SIDEBAR>

List the filesystem with

```
tree $RISCV/buildroot/output/images/disassembly/rootfs
```

</END>



```

cache -> ../tmp
lib
misc -> ../tmp

```

```

lock -> ../run/lock
log -> ../tmp
run -> ../run

```

```

spool -> ../tmp
tmp -> ../tmp
www

```

Figure 22.1 Buildroot root filesystem using BusyBox

22.2.9 Device Tree

Figure 22.2 lists the Wally device tree from `$WALLY/linux/devicetree/wally-virt.dts`. It shows that physical memory begins at address `0x80000000` and has a size of `0x10000000` = 256 MiB. There is one CPU with a clock and timer timebase frequency of `0x1C9C380` = 30. The CPU supports all the standard extensions and SV48 virtual memory with the Svadu extension. The SoC has a UART, PLIC, and CLINT at specified addresses in the memory map. Linux uses the compatible fields to determine which device drivers to use.

```

/dts-v1/;

/ {
    #address-cells = <0x02>;
    #size-cells = <0x02>;
    compatible = "wally-virt";
    model = "wally-virt,qemu";

    chosen {
        linux,initrd-end = <0x85c43a00>;
        linux,initrd-start = <0x84200000>;
        bootargs = "root=/dev/vda ro";
        stdout-path = "/soc/uart@10000000";
    };

    memory@80000000 {
        device_type = "memory";
        reg = <0x00 0x80000000 0x00 0x10000000>;
    };

    cpus {
        #address-cells = <0x01>;
        #size-cells = <0x00>;
        clock-frequency = <0x1C9C380>;
        timebase-frequency = <0x1C9C380>;

        cpu@0 {
            phandle = <0x01>;
            device_type = "cpu";
            reg = <0x00>;
            status = "okay";
            compatible = "riscv";
            riscv,isa = "rv64imafdcsv";
            riscv,isa-extensions = "svadu";
            mmu-type = "riscv,sv48";

            interrupt-controller {
                #interrupt-cells = <0x01>;
                interrupt-controller;
                compatible = "riscv,cpu-intc";
                phandle = <0x02>;
            };
        };
    };

    soc {
        #address-cells = <0x02>;

```

```

#size-cells = <0x02>;
compatible = "simple-bus";
ranges;

uart@10000000 {
    interrupts = <0x0a>;
    interrupt-parent = <0x03>;
    clock-frequency = <0x384000>;
    reg = <0x00 0x10000000 0x00 0x100>;
    compatible = "ns16550a";
};

plic@c0000000 {
    phandle = <0x03>;
    riscv,ndev = <0x35>;
    reg = <0x00 0xc0000000 0x00 0x210000>;
    interrupts-extended = <0x02 0x0b 0x02 0x09>;
    interrupt-controller;
    compatible = "sifive,plic-1.0.0\0riscv,plic0";
    #interrupt-cells = <0x01>;
    #address-cells = <0x00>;
};

clint@20000000 {
    interrupts-extended = <0x02 0x03 0x02 0x07>;
    reg = <0x00 0x20000000 0x00 0x10000>;
    compatible = "sifive,clint0\0riscv,clint0";
};
};
};

```

Figure 22.2 Wally device tree

22.2.10 Init Process

The first user process spawned by the kernel is `/sbin/init`. Like other user processes, it executes in U-mode. In Buildroot, `init` is part of BusyBox. The source code is in `buildroot/output/build/busybox-1.36.1/init/init.c`.

`init` spawns a set of processes defined in `/etc/inittab`. These processes include setting the hostname, running the `/etc/init.d` scripts to start system logging, and starting a `getty` terminal with a prompt to log in.

22.2.11 Trap Handlers

The M-mode trap handler is part of OpenSBI and is named `_trap_handler` in `fw_base.S`. The handler fetches a thread pointer from `mscratch` and uses it to set up a stack pointer, then saves the `mepc`, `mstatus`, and general registers on the stack. It invokes the `sbi_trap_handler` in `sbi_trap.c`, and then restores the registers and returns with `mret`.

The `sbi_trap_handler` processes M-mode interrupts. The handler emulates misaligned loads and stores and missing CSRs, and performs SBI calls (see Section 8.2.5.1XREF).

OpenSBI `delegate_traps` delegates S-mode interrupts (external, timer, and software) and breakpoint, U-mode `ecall`, and misaligned instruction fetches to the kernel's S-mode handler `handle_exception`, which is found in `linux-6.6/arch/riscv/kernel/entry.S`. `handle_exception` sets up a stack pointer from `sscratch` and saves `sstatus`, `sepc`, `stval`, `stcause`, `sscratch`, and general registers on the stack. It disables the FPU and U-mode page accesses to protect against accidental illegal operations. Then it processes the exception, restores the registers, and returns with `sret`.

22.2.11.1 Software Interrupts

If one user process wants to send a message to another process, it can put the message in a shared memory buffer, then send a software interrupt to that process so it knows the message is waiting. The CLINT only can set the machine software interrupt, while the OS needs to process it in S-mode, so it takes two traps to send a software interrupt and two more to receive.

The sending user-level processes use `ecall` to invoke the Linux `sbi_send_ipi(cpu)` system call. The S-mode Linux `sbi_send_ipi` function in turn makes another `ecall` to invoke the OpenSBI `sbi_send_ipi()` firmware system call. The M-mode OpenSBI `sbi_send_ipi` function sets the appropriate MSIP bit in the CLINT for the hart that should receive the software interrupt, then returns to Linux, which finally returns to the user process.

Setting MSIP in the CLINT triggers a machine software interrupt in the target hart, invoking `sbi_trap_handler`, which in turn calls `sbi_ipi_process()` to clear the MSIP bit and set `sip.SSIP`, then returns. This triggers a supervisor software interrupt, which is delegated to the S-mode exception handler, `handle_exception` routine, which invokes `do_irq`, which invokes `riscv_software_interrupt()`, which clears `sip.SSIP` and finally notifies the target process.

22.2.11.2 Timer Interrupts

The OS scheduler uses timer interrupts to determine when to switch between threads. Analogous to software interrupts, the CLINT can set `mip.MTIP`, triggering an M-mode timer interrupt, causing OpenSBI to set `sip.STIP`, causing a supervisor timer interrupt, invoking the Linux scheduler. The interval is configurable but can be as often as every millisecond, so the trap overhead saving and restoring the registers twice can be significant.

The `Sstc` extension (Section 8.2.5.2XREF) reduces this overhead by sending an S-mode timer interrupt directly to Linux without involving M-mode firmware. If the `stimecmp` register exists, the OpenSBI firmware (`sbi_hart.c`) sets `menvcfg.STCE` to enable `Sstc` and `mcounteren.TM` to allow supervisor access to the counters. Then Linux programs the `stimecmp` register with the time for the next interrupt. The timer directly sets `sip.STIP`, which triggers the supervisor timer interrupt.


```

Firmware RW Size      : 66 KB
Firmware Heap Offset  : 0x28000
Firmware Heap Size    : 34 KB (total), 2 KB (reserved), 9 KB (used), 22 KB (free)
Firmware Scratch Size : 4096 B (total), 760 B (used), 3336 B (free)
Runtime SBI Version    : 1.0

Domain0 Name          : root
Domain0 Boot HART      : 0
Domain0 HARTs         : 0*
Domain0 Region00       : 0x0000000002000000-0x000000000200ffff M: (I,R,W) S/U: ()
Domain0 Region01       : 0x0000000008000000-0x0000000008001ffff M: (R,X) S/U: ()
Domain0 Region02       : 0x00000000080020000-0x0000000008003ffff M: (R,W) S/U: ()
Domain0 Region03       : 0x0000000000000000-0xffffffff M: (R,W,X) S/U: (R,W,X)
Domain0 Next Address   : 0x00000000080200000
Domain0 Next Arg1      : 0x00000000082200000
Domain0 Next Mode      : S-mode
Domain0 SysReset       : yes
Domain0 SysSuspend     : yes

Boot HART ID          : 0
Boot HART Domain      : root
Boot HART Priv Version : v1.12
Boot HART Base ISA     : rv64imafdc
Boot HART ISA Extensions : time,sstc
Boot HART PMP Count    : 16
Boot HART PMP Granularity : 4
Boot HART PMP Address Bits: 54
Boot HART MHPM Count   : 16
Boot HART MIDELEG      : 0x0000000000001666
Boot HART MEDELEG      : 0x000000000000b509
[ 0.000000] Linux version 6.6.0 (harris@chips) (riscv64-buildroot-linux-gnu-gcc.br_real (Buildroot 2023.05.3-dirty) 12.3.0, GNU ld (GNU Binutils) 2.38) #2 Tue Nov 21 19:16:18
PST 2023
[ 0.000000] Machine model: wally-virt,qemu
[ 0.000000] SBI specification v1.0 detected
[ 0.000000] SBI implementation ID=0x1 Version=0x10003
[ 0.000000] SBI TIME extension detected
[ 0.000000] SBI IPI extension detected
[ 0.000000] SBI RFENCE extension detected
[ 0.000000] efi: UEFI not found.
[ 0.000000] OF: reserved mem: 0x00000000080000000..0x0000000008001ffff (128 KiB) nomap non-reusable mmode_resv0@80000000
[ 0.000000] OF: reserved mem: 0x00000000080020000..0x0000000008003ffff (128 KiB) nomap non-reusable mmode_resv1@80020000
[ 0.000000] Zone ranges:
[ 0.000000] DMA32 [mem 0x00000000080000000-0x000000000800ffff]
[ 0.000000] Normal empty
[ 0.000000] Movable zone start for each node
[ 0.000000] Early memory node ranges
[ 0.000000] node 0: [mem 0x00000000080000000-0x0000000008003ffff]
[ 0.000000] node 0: [mem 0x00000000080040000-0x000000000800ffff]
[ 0.000000] Initmem setup node 0 [mem 0x00000000080000000-0x000000000800ffff]
[ 0.000000] Falling back to deprecated "riscv,isa"
[ 0.000000] riscv: base ISA extensions acdfim
[ 0.000000] riscv: ELF capabilities acdfim
[ 0.000000] Kernel command line: root=/dev/vda ro
[ 0.000000] Dentry cache hash table entries: 32768 (order: 6, 262144 bytes, linear)
[ 0.000000] Inode-cache hash table entries: 16384 (order: 5, 131072 bytes, linear)
[ 0.000000] Built 1 zonelists, mobility grouping on. Total pages: 64640
[ 0.000000] mem auto-init: stack:off, heap alloc:off, heap free:off
[ 0.000000] Virtual kernel memory layout:
[ 0.000000] fixmap : 0xff1bffffea00000 - 0xff1bffff000000 (6144 kB)
[ 0.000000] pci io : 0xff1bffff0000000 - 0xff1c000000000000 ( 16 MB)
[ 0.000000] vmemmap : 0xff1c0000000000000 - 0xff200000000000000 (1024 TB)
[ 0.000000] vmlalloc : 0xff200000000000000 - 0xff600000000000000 (16384 TB)
[ 0.000000] modules : 0xffffffff046e8000 - 0xffffffff800000000 (1977 MB)
[ 0.000000] lowmem : 0xff600000000000000 - 0xff600000100000000 ( 256 MB)
[ 0.000000] kernel : 0xffffffff800000000 - 0xffffffffffffff (2047 MB)
[ 0.000000] Memory: 123724K/262144K available (2542K kernel code, 3693K rwdata, 2048K rodata, 62517K init, 249K bss, 138420K reserved, 0K cma-reserved)
[ 0.000000] SLUB: HWalign=64, Order=0-3, MinObjects=0, CPUs=1, Nodes=1
[ 0.000000] NR_IRQS: 64, nr_irqs: 64, preallocated irq: 0
[ 0.000000] riscv-intc: 64 local interrupts mapped
[ 0.000000] plic@c000000: mapped 53 interrupts with 1 handlers for 2 contexts.
[ 0.000000] clocksource: riscv_clocksource: mask: 0xffffffff max_cycles: 0xdd67c8a60, max_idle_ns: 881590406601 ns
[ 0.000016] sched_clock: 64 bits at 30MHz, resolution 33ns, wraps every 4398046511088ns
[ 0.000809] Console: colour dummy device 80x25
[ 0.000865] printk: console [tty0] enabled
[ 0.001425] Calibrating delay loop (skipped), value calculated using timer frequency.. 60.00 BogoMIPS (lpj=120000)
[ 0.001454] pid_max: default: 32768 minimum: 301
[ 0.001624] Mount-cache hash table entries: 512 (order: 0, 4096 bytes, linear)
[ 0.001634] Mountpoint-cache hash table entries: 512 (order: 0, 4096 bytes, linear)
[ 0.005643] riscv: ELF compat mode supported
[ 0.005708] ASID allocator using 16 bits (65536 entries)
[ 0.005894] EFI services will not be available.
[ 0.007213] devtmpfs: initialized
[ 0.008161] clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 7645041785100000 ns
[ 0.008210] futex hash table entries: 256 (order: 1, 12288 bytes, linear)
[ 0.009045] DMA: preallocated 128 KiB GFP_KERNEL pool for atomic allocations
[ 0.009101] DMA: preallocated 128 KiB GFP_KERNEL|GFP_DMA32 pool for atomic allocations
[ 0.032655] cpu0: Ratio of byte access time to unaligned word access is 8.33, unaligned accesses are fast

```

```
[ 0.034975] clocksource: Switched to clocksource riscv_clocksource
[ 0.043042] workingset: timestamp_bits=62 max_order=15 bucket_order=0
[ 0.043545] io scheduler mq-deadline registered
[ 0.043562] io scheduler kyber registered
[ 0.043596] io scheduler bfq registered
[ 0.064218] Serial: 8250/16550 driver, 1 ports, IRQ sharing disabled
[ 0.067933] 100000000.uart: ttyS0 at MMIO 0x10000000 (irq = 2, base_baud = 230400) is a 16550A
[ 0.068242] printk: console [ttyS0] enabled
[ 0.082181] debug_vm_pgtable: [debug_vm_pgtable]: Validating architecture page table helpers
[ 0.087038] clk: Disabling unused clocks
[ 0.107776] Unpacking initramfs...
[ 0.163386] Freeing initrd memory: 60388K
[ 0.175992] Freeing unused kernel image (initmem) memory: 62516K
[ 0.176178] Run /init as init process
init started: BusyBox v1.36.1 (2023-11-21 19:10:20 PST)
Starting syslogd: OK
Starting klogd: OK
Running sysctl: OK
Saving 256 bits of non-creditable seed for next boot
Starting network: /etc/init.d/S40network: line 12: /sbin/ifup: not found
FAIL
```

BusyBox v1.36.1 (2023-11-21 19:10:20 PST) built-in shell (ash)
Enter 'help' for a list of built-in commands.

Hello this ~/profile is meant to demonstrate running some basic commands on Wally.

I am root

And I am on WallyHostname

Created myFile.txt and moved it to myDir. It contains:

Hello World!

And farewell!

Created myScript.sh. Running it yields:

Hello this is another example script

Created symLinkToMyScript.sh. Running it yields:

Hello this is another example script

Now let's remove all these example files and scripts

Here is disk usage:

Filesystem	Size	Used	Available	Use%	Mounted on
devtmpfs	60.4M	0	60.4M	0%	/dev
tmpfs	120.4M	0	120.4M	0%	/dev/shm
tmpfs	120.4M	0	120.4M	0%	/tmp
tmpfs	120.4M	8.0K	120.4M	0%	/run

And here are the current processes:

PID	USER	VSZ	STAT	COMMAND
1	root	2208	S	init
2	root	0	SW	[kthreadd]
3	root	0	SW	[pool_workqueue_]
4	root	0	IW<	[kworker/R-slab_]
5	root	0	IW	[kworker/0:0-eve]
6	root	0	IW<	[kworker/0:0H]
7	root	0	IW	[kworker/u2:0-ev]
8	root	0	IW<	[kworker/R-mm_pe]
9	root	0	SW	[ksoftirqd/0]
10	root	0	SW	[kdevtmpfs]
11	root	0	SW	[khungtaskd]
12	root	0	SW	[oom_reaper]
13	root	0	IW<	[kworker/R-write]
14	root	0	SW	[kcompactd0]
15	root	0	IW<	[kworker/R-kbloc]
16	root	0	IW	[kworker/0:1-eve]
17	root	0	SW	[kswapd0]
18	root	0	IW	[kworker/u2:1-ev]
19	root	0	IW	[kworker/u2:2-ev]
40	root	2208	S	/sbin/klogd -n
54	root	2208	S	~/bin/sh
70	root	2208	R	ps

And finally a login prompt.

WallyHostname login:

WallyHostname login: root

login[71]: root login on 'console'

BusyBox v1.36.1 (2023-11-21 19:10:20 PST) built-in shell (ash)
Enter 'help' for a list of built-in commands.

```
# mkdir test
# ls
test
```

Figure 22.3 Linux boot messages

To view the physical addresses of the boot files from Table 22.3, press <Ctrl-a> c to enter the QEMU monitor. Then type info roms

```
(qemu) info roms
addr=0000000000001000 size=0x000028 mem=rom name="mrom.reset"
addr=0000000000001028 size=0x000030 mem=rom name="mrom.info"
addr=0000000080000000 size=0x01f7d8 mem=ram name="fw_jump.elf ELF program header segment 1"
addr=0000000080020000 size=0x005058 mem=ram name="fw_jump.elf ELF program header segment 2"
addr=0000000080200000 size=0x46a8c00 mem=ram name="Image"
addr=0000000088200000 size=0x3af9c00 mem=ram name="rootfs.cpio"
addr=000000008fe00000 size=0x0005ff mem=ram name="fdt"
(qemu) quit
```

22.3.2 Booting Linux on Wally in Questa Simulation

Launch a simulation of Buildroot on Wally in Questa using `wsim`.

```
$ wsim buildroot buildroot --args "+INSTR_LIMIT=600000000" --lockstep
```

Linux does not naturally terminate, so the `INSTR_LIMIT` argument specifies how many instructions to run. 1,400,000 is enough to reach the OpenSBI banner, and 600 million is more than enough to reach the login prompt. Use the `--lockstep` flag to optionally compare against the ImperasDV model using the lock-step-compare methodology described in Section 5.7.5XREF.

The Linux boot presently executes about 492 million instructions and takes about 40 hours on a high-performance PC with lockstep in Questa, or 2 hours in Verilator without lockstep. Linux continues to evolve and your mileage may vary.

22.3.3 Modifying Buildroot

Advanced users may wish to modify Buildroot. The only source files manually given to buildroot are located in `$WALLY/linux/buildroot-config-src/wally`. Some of these source files are configuration files, and others are files meant to go in the root filesystem of the final machine.

22.3.3.1 Root File System Overlay

`$WALLY/linux/buildroot-config-src/wally/rootfs_overlay` contains files that are meant to go in the root filesystem of the final machine. These files aren't strictly necessary for booting Linux but can be helpful for testing purposes. For example, `/etc/inittab` is modified from the default version to cause the system to login and start up a shell without any user input for username or password, as shown in blue in Figure 22.4. The `/.profile` file contains some test commands executed before initiating an actual login prompt, as shown in Figure 22.5. This way, shell commands can be tested without requiring user input (which may cause complications in QEMU record/replay mode).

```
# /etc/inittab
#
# Copyright (C) 2001 Erik Andersen <andersen@codepoet.org>
```



```

#
# Note: BusyBox init doesn't support runlevels. The runlevels field is
# completely ignored by BusyBox init. If you want runlevels, use
# sysvinit.
#
# Format for each entry: <id>:<runlevels>:<action>:<process>
#
# id      == tty to run on, or empty for /dev/console
# runlevels == ignored
# action  == one of sysinit, respawn, askfirst, wait, and once
# process == program to run

# Startup the system
::sysinit:/bin/mount -t proc proc /proc
::sysinit:/bin/mount -o remount,rw /
::sysinit:/bin/mkdir -p /dev/pts /dev/shm
::sysinit:/bin/mount -a
::sysinit:/sbin/swapon -a
null::sysinit:/bin/ln -sf /proc/self/fd /dev/fd
null::sysinit:/bin/ln -sf /proc/self/fd/0 /dev/stdin
null::sysinit:/bin/ln -sf /proc/self/fd/1 /dev/stdout
null::sysinit:/bin/ln -sf /proc/self/fd/2 /dev/stderr
::sysinit:/bin/hostname -F /etc/hostname
# now run any rc scripts
::sysinit:/etc/init.d/rcS

# (commented out) Put a getty on the serial port
#console::respawn:/sbin/getty -L console 0 vt100 # GENERIC_SERIAL
# Actually no, let's automatically login
console::respawn:/bin/sh

# Stuff to do for the 3-finger salute
#::ctrlaltdel:/sbin/reboot

# Stuff to do before rebooting
::shutdown:/etc/init.d/rcK
::shutdown:/sbin/swapoff -a
::shutdown:/bin/umount -a -r

```

Figure 22.4 /etc/inittab modified to login automatically

```

echo "Hello this ~/.profile is meant to demonstrate running some basic commands on Wally."
echo "I am $(whoami)"
echo "And I am on $(hostname)"
touch myFile.txt
echo "Hello World!" > myFile.txt
echo "And farewell!" >> myFile.txt
mkdir myDir
mv myFile.txt myDir
echo "Created myFile.txt and moved it to myDir. It contains:"
cat myDir/myFile.txt
touch myScript.sh
echo "echo \"Hello this is another example script\"" > myScript.sh
chmod +x myScript.sh
echo "Created myScript.sh. Running it yields:"
./myScript.sh
cd myDir
ln -s ../myScript.sh symLinkToMyScript.sh
echo "Created symLinkToMyScript.sh. Running it yields:"
./symLinkToMyScript.sh
echo "Now let's remove all these example files and scripts"
cd /
rm -r myDir
rm myScript.sh
echo "Here is disk usage:"
df -h

```

```
echo "And here are the current processes:"
ps
echo "And finally a login prompt."
Login
```

Figure 22.5 /.profile

22.3.3.2 Configuration Files

Buildroot provides an interactive menuconfig interface for configuration, consisting of three files and config tools: main.config (menuconfig), linux.config (linux-menuconfig), busybox.config (busybox-menuconfig). For example, our configuration was edited to enable debug symbols. To edit main.config using menuconfig:

```
$ sudo su cad
$ cd $RISCV/buildroot
$ make menuconfig
```

Running the linux Makefile from Section 22.2.1 automatically copied main.config and linux.config to the \$RISCV/buildroot/board/wally directory and then copied \$RISCV/buildroot/board/wally/main.config to \$RISCV/buildroot/.config. \$RISCV/buildroot/board/wally/linux.config was copied to \$RISCV/buildroot/output/build/linux-<version>/.config

make menuconfig automatically loads \$RISCV/buildroot/.config and saves to the same file. Similarly, make linux-menuconfig and make busybox-menuconfig edit linux.config and busybox.config, respectively. All the config files may also be changed with a text editor.

22.3.3.3 Rebuilding the Image

After making changes to the root filesystem or configuration files, rebuild Buildroot with the command.

```
$ cd $WALLY/linux
$ make Image BUILDROOT=$(RISCV)/buildroot
$ make disassemble BUILDROOT=$(RISCV)/buildroot
$ make dumptvs
```

22.4 Summary

Linux is an open-source industrial-strength OS widely used in everything from embedded systems to mobile devices to PCs to servers. This chapter described the hardware and software needed to boot Linux on Wally.

The OS components include a zero stage bootloader (ZSBL), firmware, device tree, kernel, and root file system. They may be read from an SD card into RAM on an FPGA board or simply initialized in RAM during logic simulation.

On reset, the processor executes the ZSBL from a small boot ROM. The ZSBL reads the firmware and device tree from the SD card if necessary, then jumps to the OpenSBI firmware bootloader in the firmware.

The OpenSBI firmware bootloader runs in M-mode. It sets up an M-mode trap handler, reads platform-specific information from the device tree, initializes harts, peripherals, and CSRs, and prints initial boot messages to the COM port. It then loads the kernel and enters S-mode to begin executing the kernel.

The Linux kernel initializes a virtual memory page table and turns on virtual memory. Henceforth, Linux runs in S-mode with virtual addressing. It sets up an S-mode trap handler. It then initializes many services including memory management, scheduling, timers, and console I/O. It reads the root file system, and calls the `init` process, which is the first U-mode process and the parent of all other Linux processes. Finally, the user sees the login prompt on the console.

The cvw repository supports booting Buildroot Linux on three platforms. First, a generic RISC-V processor can be emulated using QEMU. Second, a logic simulator (Questa, VCS, or Verilator) simulates Wally boot OpenSBI and the Linux kernel. Finally, Chapter 23 implements Wally on real hardware, an FPGA, and boots Linux into an interactive terminal shell.